Projet : Smart Healthcare Data Space

1. Contexte

Dans le domaine de la santé, les hôpitaux disposent de données médicales sensibles qu'ils ne peuvent pas partager librement. Les espaces de données (Data Spaces) permettent une collaboration sécurisée entre établissements tout en garantissant la confidentialité des patients. Ce projet vise à concevoir un prototype démontrant cette approche en utilisant le **Federated Learning**, **Eclipse Dataspace Connector (EDC)** et **Kubernetes**.

2. Objectif

Développer, en **4 semaines**, une plateforme distribuée permettant à **trois hôpitaux** de collaborer pour entraîner un modèle d'intelligence artificielle sans partager directement leurs données.

3. Technologies imposées

- ✓ Python
- ✓ TensorFlow ou PyTorch
- ✓ Flower (Federated Learning)
- ✓ Eclipse Dataspace Connector (EDC)
- ✓ Docker
- ✓ Kubernetes (Minikube ou Kind)
- ✓ Git
- ✓ FastAPI
- ✓ PostgreSQL (optionnel)

4. Fonctionnalités attendues

Module 1 : Gestion des données

- Création de trois jeux de données représentant trois hôpitaux.
- Les données restent stockées localement.
- Chargement automatique des données.

Module 2 : Federated Learning

- Entraînement local sur chaque hôpital.
- Coordination via un serveur Flower.
- Agrégation des modèles (FedAvg).
- Affichage de :
- Accuracy
- Loss
- Nombre de rounds.

Module 3 : Data Space (EDC)

- Déployer un connecteur EDC pour chaque hôpital.
- Publication d'un dataset.

- Découverte d'un dataset.
- Simulation d'un contrat d'échange.
- Journalisation des échanges.

Module 4 : Déploiement Kubernetes

- Déployer sous Kubernetes :
- Serveur Flower
- Trois clients FL
- Trois connecteurs EDC
- API FastAPI
- Tableau de bord

Module 5 : Tableau de bord

- Afficher :
- nombre d'hôpitaux connectés ;
- état des connecteurs EDC ;
- progression de l'entraînement ;
- précision du modèle ;
- temps d'exécution.

5. Organisation du projet (Planning sur 1 mois)

Semaine 1 : Conception

Objectifs

- Étudier les technologies.
- Concevoir l'architecture.
- Préparer les datasets.
- Installer l'environnement.

Livrables

- Diagramme d'architecture.
- Dépôt Git.
- Environnement Docker opérationnel.

Semaine 2 : Développement

Objectifs

- Implémenter le Federated Learning.
- Développer les API.
- Tester l'entraînement distribué.

Livrables

- Prototype FL fonctionnel.
- Premier entraînement distribué.

Semaine 3 : Intégration

Objectifs

- Déployer les connecteurs EDC.
- Connecter les différents services.
- Déployer sous Kubernetes.

Livrables

- Cluster Kubernetes opérationnel.
- Communication entre les services.

Semaine 4 : Validation

Objectifs

- Réaliser les tests.
- Corriger les anomalies.
- Préparer la démonstration.
- Finaliser la documentation.

Livrables

- Démonstration finale.
- Rapport.
- Présentation.

Industrial Data Space for Predictive Maintenance

1. Contexte

Les usines modernes sont équipées de nombreux capteurs IoT permettant de surveiller en temps réel l'état des machines (température, vibrations, consommation électrique, pression, etc.). Ces données constituent une source précieuse pour mettre en œuvre des solutions de **maintenance prédictive** afin d'anticiper les pannes et de réduire les coûts de maintenance.

Cependant, les entreprises sont souvent réticentes à partager directement leurs données industrielles avec d'autres usines ou partenaires. Les **Data Spaces** permettent de partager les données de manière sécurisée tout en conservant leur contrôle. Couplés à un **Machine Learning distribué**, ils offrent une solution collaborative pour améliorer les modèles de prédiction sans centraliser les données.

2. Objectif du projet

Développer un prototype d'**Industrial Data Space** permettant à plusieurs usines de collaborer pour améliorer un modèle de maintenance prédictive grâce au partage sécurisé des données et à l'apprentissage distribué.

3. Technologies imposées

- ✓ Python
- ✓ IoT (simulation de capteurs ou Node-RED/MQTT)
- ✓ Eclipse Dataspace Connector (EDC)
- ✓ Flower ou TensorFlow Federated
- ✓ Scikit-learn, TensorFlow ou PyTorch
- ✓ Docker
- ✓ Kubernetes (Minikube ou Kind)
- ✓ Git
- ✓ FastAPI (optionnel)

4. Fonctionnalités attendues

Module 1 : Simulation IoT

Le système devra simuler plusieurs usines équipées de capteurs IoT.

Chaque usine devra générer des données telles que :

- Température
- Vibrations
- Pression
- Humidité
- Consommation électrique
- État de fonctionnement (normal ou panne)

Les données seront stockées localement dans chaque usine.

Module 2 : Industrial Data Space

Chaque usine devra disposer d'un **Data Space Connector (EDC)** permettant de :

- publier un jeu de données ;
- découvrir les jeux de données disponibles ;
- négocier un contrat d'échange ;
- autoriser les accès selon des politiques définies ;
- journaliser les échanges entre partenaires.

Module 3 : Machine Learning distribué

Développer un modèle de maintenance prédictive capable de :

- entraîner localement un modèle sur les données de chaque usine ;
- agréger les modèles via un serveur de fédération (FedAvg) ;
- mettre à jour le modèle global ;
- afficher les performances (Accuracy, Precision, Recall, F1-score).

Module 4 : Déploiement Kubernetes

Déployer les principaux services sous Kubernetes :

- serveur de fédération ;
- clients Machine Learning ;
- connecteurs EDC ;
- API ou tableau de bord.

Module 5 : Tableau de bord

Développer une interface permettant d'afficher :

- les usines connectées ;
- l'état des connecteurs EDC ;
- les données IoT en temps réel (ou simulées) ;
- les performances du modèle de prédiction ;
- les alertes de maintenance prédictive.

5. Planning (1 mois)

Semaine 1 : Analyse et conception

Objectifs

- Comprendre le fonctionnement des Data Spaces.
- Concevoir l'architecture globale.
- Simuler les données IoT.
- Préparer les jeux de données.

Livrables

- Architecture générale.
- Dépôt Git.
- Données IoT simulées.
- Répartition des tâches.

Semaine 2 : Développement

Objectifs

- Développer le simulateur IoT.
- Implémenter le modèle de Machine Learning.
- Réaliser les premiers entraînements distribués.

Livrables

- Prototype fonctionnel.
- Premier modèle entraîné.
- Documentation technique intermédiaire.

Semaine 3 : Intégration

Objectifs

- Déployer les connecteurs EDC.

- Intégrer le Machine Learning distribué.
- Déployer les services avec Kubernetes.

Livrables

- Cluster Kubernetes opérationnel.
- Communication entre les différentes usines.
- Démonstration intermédiaire.

Semaine 4 : Validation

Objectifs

- Tester le système complet.
- Évaluer les performances.
- Corriger les anomalies.
- Préparer la démonstration finale.

Livrables

- Rapport final.
- Présentation.
- Démonstration du prototype.

Smart Mobility Data Space

1. Contexte

Les villes intelligentes (Smart Cities) génèrent quotidiennement d'importants volumes de données provenant de différents acteurs : transports publics, véhicules connectés, capteurs routiers, services de stationnement, vélos et trottinettes en libre-service. Ces données sont généralement réparties entre plusieurs organismes et ne sont pas facilement partageables.

Les **Data Spaces** offrent une solution pour permettre un partage sécurisé et contrôlé des données entre ces acteurs. En s'appuyant sur les principes de **Gaia-X**, des **APIs distribuées** et de l'**Edge AI**, il devient possible de développer des services innovants tout en garantissant la souveraineté des données.

2. Objectif du projet

Développer un prototype de **Smart Mobility Data Space** permettant à plusieurs acteurs de la mobilité urbaine de partager leurs données de manière sécurisée afin d'améliorer les services de mobilité (gestion du trafic, prévision des embouteillages, optimisation des itinéraires, disponibilité des parkings, etc.).

3. Technologies imposées

- Python
- FastAPI (APIs distribuées)
- Gaia-X (principes et architecture)
- Edge AI (TensorFlow Lite, ONNX ou équivalent)
- Docker
- Kubernetes (Minikube ou Kind)
- Git
- PostgreSQL ou MongoDB (optionnel)

4. Fonctionnalités attendues

Module 1 : Simulation des données de mobilité

Le système devra simuler plusieurs fournisseurs de données, par exemple :

- Réseau de bus
- Métro ou tramway
- Capteurs de circulation

- Parkings intelligents
- Vélos ou trottinettes en libre-service

Chaque fournisseur conserve ses données localement.

Les données pourront inclure :

- vitesse moyenne
- densité du trafic
- taux d'occupation des parkings
- horaires des transports
- incidents de circulation
- géolocalisation

Module 2 : APIs distribuées

Chaque fournisseur devra exposer une API REST permettant de :

- publier des données ;
- consulter des données autorisées ;
- rechercher des jeux de données ;
- contrôler les accès via des politiques simples ;
- journaliser les requêtes.

Les APIs devront être documentées avec **OpenAPI/Swagger**.

Module 3 : Data Space Gaia-X

Le prototype devra respecter les principes fondamentaux de Gaia-X :

- souveraineté des données ;
- contrôle des accès ;
- partage sécurisé entre partenaires ;
- gestion des politiques d'utilisation des données ;
- traçabilité des échanges.

Une simulation de catalogue de services ou de partenaires pourra être réalisée.

Module 4 : Edge AI

Développer un modèle d'intelligence artificielle exécuté en périphérie (Edge) permettant, par exemple :

- la prédiction des embouteillages ;
- l'estimation du temps de trajet ;
- la prédiction de la disponibilité des parkings ;
- la détection d'anomalies dans le trafic.

Chaque nœud Edge effectue localement ses prédictions avant de partager uniquement les résultats utiles.

Module 5 : Tableau de bord

Développer une interface permettant de visualiser :

- les fournisseurs connectés ;
- l'état des APIs ;
- les données de mobilité simulées ;
- les prédictions Edge AI ;
- les indicateurs de trafic ;
- les échanges réalisés entre partenaires.

5. Planning (1 mois)

Semaine 1 : Analyse et conception

Objectifs

- Étudier les concepts de Gaia-X et des Data Spaces.
- Définir les acteurs et les cas d'utilisation.
- Concevoir l'architecture globale.
- Mettre en place l'environnement de développement.

Livrables

- Cahier d'architecture.
- Diagramme des composants.
- Dépôt Git.
- Répartition des tâches.

Semaine 2 : Développement

Objectifs

- Développer les APIs distribuées.

- Simuler les fournisseurs de données.
- Mettre en place les échanges entre les services.

Livrables

- APIs REST fonctionnelles.
- Documentation Swagger/OpenAPI.
- Prototype de partage des données.

Semaine 3 : Intégration

Objectifs

- Intégrer les principes Gaia-X.
- Développer le modèle Edge AI.
- Déployer les services avec Docker et Kubernetes.

Livrables

- Prototype intégré.
- Services déployés.
- Tests des échanges sécurisés.

Semaine 4 : Validation

Objectifs

- Tester le fonctionnement global.
- Évaluer les performances du modèle Edge AI.
- Corriger les anomalies.
- Préparer la démonstration finale.

Livrables

- Rapport final.
- Présentation.
- Démonstration fonctionnelle.

Energy Data Space

1. Contexte

La transition énergétique repose sur une meilleure coordination entre les différents acteurs du réseau électrique : producteurs d'énergie, gestionnaires de réseau et consommateurs. Les **Smart Grids** permettent de collecter et d'échanger en temps réel des données de consommation et de production afin d'optimiser l'utilisation des ressources énergétiques.

Cependant, ces données sont souvent distribuées entre plusieurs organisations et leur partage est soumis à des contraintes de confidentialité, de sécurité et de gouvernance. Les **Energy Data Spaces** offrent un cadre de partage sécurisé, tandis que les **Federated Analytics** permettent d'obtenir des indicateurs globaux sans centraliser les données. Une **Data Governance** adaptée garantit la qualité, la traçabilité et le respect des règles d'accès aux données.

2. Objectif du projet

Développer un prototype d'**Energy Data Space** permettant à plusieurs producteurs et consommateurs d'énergie de partager des données de manière sécurisée afin d'analyser et d'optimiser la consommation énergétique tout en préservant la souveraineté des données.

3. Technologies imposées

- Python
- Smart Grid (simulation de compteurs intelligents)
- Federated Analytics
- Data Governance
- FastAPI
- Docker
- Kubernetes (Minikube ou Kind)
- Git
- PostgreSQL ou MongoDB (optionnel)

4. Fonctionnalités attendues

Module 1 : Simulation du Smart Grid

Le système devra simuler plusieurs acteurs :

- Producteur d'énergie
- Gestionnaire du réseau

- Entreprise consommatrice
- Habitation intelligente

Chaque acteur génère et conserve localement ses données.

Les données simulées pourront inclure :

- production énergétique (kWh)
- consommation énergétique (kWh)
- puissance instantanée
- coût de l'énergie
- taux d'utilisation des équipements
- émissions estimées de CO_2

Module 2 : Federated Analytics

Mettre en œuvre une solution permettant de calculer des statistiques globales sans transférer les données brutes.

Le système devra permettre de :

- calculer la consommation moyenne ;
- calculer la consommation totale ;
- identifier les pics de consommation ;
- comparer les performances énergétiques entre plusieurs sites ;
- produire des indicateurs agrégés.

Les calculs devront être réalisés localement puis agrégés au niveau central.

Module 3 : Data Governance

Le prototype devra intégrer des mécanismes de gouvernance des données permettant de :

- définir des rôles (producteur, consommateur, administrateur) ;
- gérer les droits d'accès aux données ;
- assurer la traçabilité des échanges ;
- contrôler la qualité et la cohérence des données ;
- appliquer des politiques simples de partage des données.

Module 4 : APIs et partage des données

Développer des APIs REST permettant :

- de publier des indicateurs énergétiques ;
- de consulter des statistiques autorisées ;
- d'effectuer des requêtes sur les données agrégées ;
- de journaliser les échanges entre les acteurs.

Les APIs devront être documentées avec **OpenAPI/Swagger**.

Module 5 : Tableau de bord

Développer une interface permettant de visualiser :

- les producteurs et consommateurs connectés ;
- la production et la consommation d'énergie en temps réel (ou simulées) ;
- les statistiques issues des Federated Analytics ;
- les alertes de surconsommation ;
- les indicateurs environnementaux (CO_2 estimé, économies d'énergie) ;
- l'état des politiques de gouvernance appliquées.

5. Planning (1 mois)

Semaine 1 : Analyse et conception

Objectifs

- Étudier les concepts de Smart Grids, Federated Analytics et Data Governance.
- Définir les acteurs et les cas d'utilisation.
- Concevoir l'architecture du système.
- Mettre en place l'environnement de développement.

Livrables

- Diagramme d'architecture.
- Modèle de données.
- Dépôt Git.
- Répartition des tâches.

Semaine 2 : Développement

Objectifs

- Simuler les producteurs et consommateurs d'énergie.
- Développer les APIs REST.

- Implémenter les calculs locaux des indicateurs énergétiques.

Livrables

- Prototype de simulation.
- APIs documentées.
- Premiers indicateurs énergétiques.

Semaine 3 : Intégration

Objectifs

- Intégrer les mécanismes de Federated Analytics.
- Implémenter les règles de Data Governance.
- Déployer les services avec Docker et Kubernetes.

Livrables

- Prototype intégré.
- Services déployés.
- Tests des échanges et des calculs fédérés.

Semaine 4 : Validation

Objectifs

- Tester le fonctionnement global.
- Vérifier les règles de gouvernance.
- Évaluer les performances des analyses fédérées.
- Préparer la démonstration finale.

Livrables

- Rapport final.
- Présentation.
- Démonstration fonctionnelle.